설비 고장을 예측하는 AI 데이터 분석

NumPy 기초

배열 생성과 구조

posco

CONTENTS

목차

PART 01 **배열 생성** NumPy 개념과 배열 만들기 · 개념·프레임

PART 02 **배열 구조와 속성** 속성 확인과 형태 변환 · 개념·프레임

NumPy를 배우는 이유

설비 센서 데이터는 하루에 수백만 개씩 쌓이는 숫자 묶음

NumPy는 이 숫자 묶음을 빠르게 계산하는 데이터 분석의 출발점

공장 설비 한 대에는 온도·진동·압력 센서가 붙어 1초에도 여러 번 값을 쏟아냅니다.

설비가 수십 대면 하루에 수백만 개의 숫자가 쌓이고, 사람이 직접 계산할 수는 없습니다.

그래서 컴퓨터에게 맡기는데, 그 가장 기본 도구가 바로 **NumPy**입니다.

앞으로 배울 **Pandas**나 **그래프 도구**도 모두 NumPy 위에서 돌아갑니다.

NumPy의 위치 — 데이터 분석 도구

NumPy는 모든 데이터 분석 도구의 기초

도구	역할	비고
NumPy	숫자 묶음 계산	가장 기초
Pandas	표 데이터 처리	NumPy 기반
Matplotlib	그래프 시각화	NumPy 기반

리스트와 배열 — 무엇이 다른가

리스트는 숫자에 곱셈하면 계산이 아니라 반복 — 배열은 묶음 전체에 한 줄로 적용

항목	리스트	배열
전체 계산	반복문 필요	한 줄로
처리 속도	느림	빠름
통계 계산	번거로움	함수 하나

리스트 자체가 느린 것이 아니라 값을 하나씩 도는 반복문이 느린 것 — 배열도 반복문으로 하나씩 돌면 이 이점이 사라짐

배열(ndarray)이란 무엇인가

posco

> 배열은 **같은 자료형**의 값만 담는다 — 그래서 빠르다
>
> 배열은 **정해진 형태**를 가진다 —한 줄 또는 표 모양

NumPy에서 데이터를 담는 그릇을 **배열**, 정확히는 **ndarray**라고 부릅니다.

배열은 숫자면 전부 숫자, **같은 종류만 담습니다**.

덕분에 메모리에 차곡차곡 정렬되어 계산이 빨라집니다.

계란판처럼 같은 크기 계란만 정해진 자리에 담겨, 빠르게 찾을 수 있습니다.

NumPy 불러오기

posco

코드 맨 위에서 NumPy를 **np라는 별명**으로 불러오기

CODE

```
import numpy as np

print(np.array([1, 2, 3]))
# 출력: [1 2 3]
```

import numpy는 불러온다는 뜻, **as np**는 앞으로 np라는 짧은 별명으로 부르겠다는 뜻.

전 세계 분석가가 똑같이 쓰는 약속 — 다른 분석 도구도 같은 방식이며, 새 코드는 항상 이 줄부터 시작.

np.array로 배열 만들기

posco

리스트를 **np.array**에 넣으면 배열이 됨 — 묶음 계산을 한 줄로

CODE

```
temp = np.array([70.5, 69.8, 73.7])
print(temp)
# 출력: [70.5 69.8 73.7]
print(temp + 5)
# 출력: [75.5 74.8 78.7]
```

출력을 보면 리스트와 달리 **쉼표 없이 공백**으로 값이 구분됩니다.

대괄호로 묶어 np.array에 전달 — temp + 5에서 모든 값에 한꺼번에 5가 더해지고 반복문이 전혀 없습니다.

배열의 자료형은 자동으로 정해짐

posco

> 모두 정수면 **정수 배열**, 하나라도 소수점 있으면 **실수 배열**
>
> 배열은 같은 자료형만 담으므로 더 넓은 쪽으로 자동 통일

np.array는 우리가 넣은 값을 보고 **자료형을 스스로 정합니다**.

모두 정수면 정수 배열이 되지만, **1, 2, 3.0**처럼 소수점이 하나라도 섞이면 전체가 실수 배열이 됩니다.

배열은 같은 자료형만 담을 수 있어서, 더 넓게 담을 수 있는 **실수 쪽으로 통일**합니다.

→ 만든 배열의 dtype이 예상과 다를 때 떠올릴 규칙

이 단원의 데이터 — CNC 가공(MCT)

공구를 회전시켜 금속을 깎는 정밀가공 설비 — 한 번의 가공이 한 줄의 데이터

실습에 쓰는 측정값은 이 설비에서 실제로 잰 수치

한 번의 가공 = 한 줄의 데이터



기록

회전수
스핀들이 1분에 몇 바퀴 도는가
실측 58 ~ 4987

토크
공구가 소재를 깎는 데 드는 힘
실측 1 ~ 147

전류
스핀들 모터가 끌어 쓰는 전류
실측 14.98 ~ 64.48

온도
가공 중 주축 주변의 온도
실측 29 ~ 31

사이클타임
한 번의 가공에 걸린 시간
실측 15554 ~ 21207

출처: KAMP 제조AI 데이터셋 — MCT 가공 Tool 관리 (중소기업기술정보진흥원·KAIST). 교육용 발췌 · 40행 6열

다섯 가지 측정값이 뜻하는 것

숫자가 무엇을 재고 있는지 알아야 결과를 해석할 수 있음

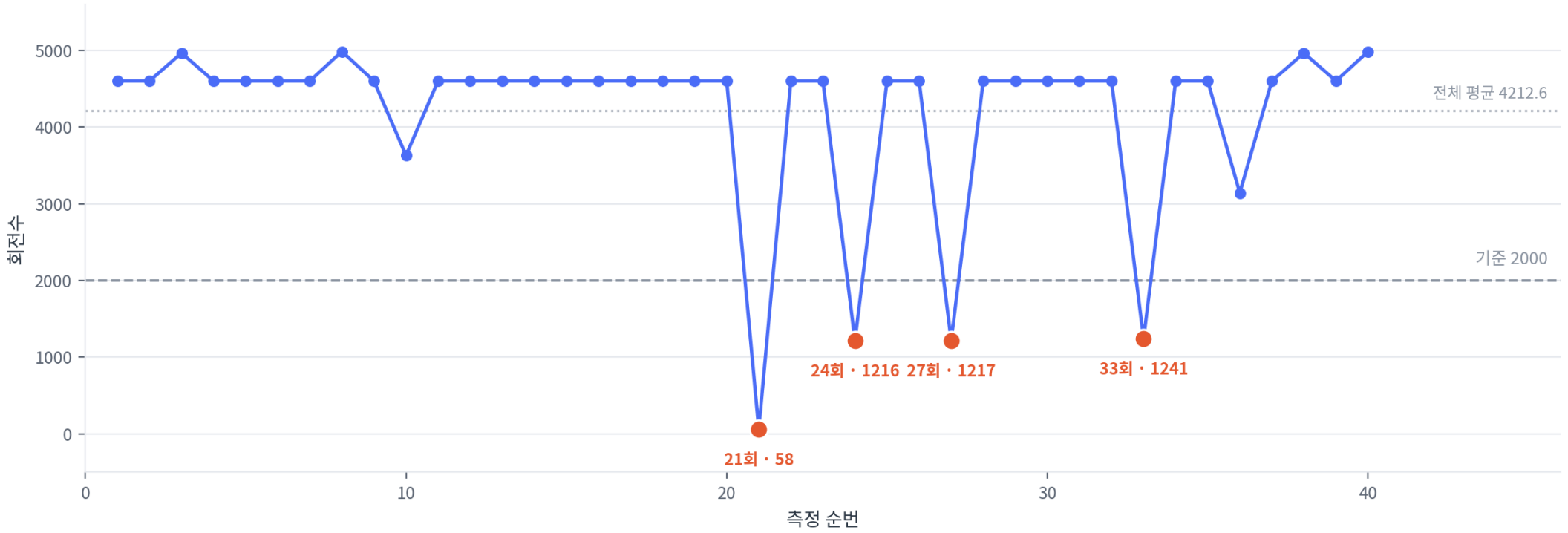
측정값	무엇을 재는가	이 단원에서의 쓰임
회전수	스핀들이 1분에 도는 바퀴 수	이상 시점 판별의 주 지표
토크	공구가 소재를 깎는 데 드는 힘	회전수와 함께 보는 보조 지표
전류	스핀들 모터가 끌어 쓰는 전류	배열 만들기·자료형 실습 소재
온도	가공 중 주축 주변의 온도	묶음 계산 실습 소재
사이클타임	한 번의 가공에 걸린 시간	모양 바꾸기 실습 소재

값이 세 종류뿐인 온도, 한 종류뿐인 공구번호처럼 분석에 쓰기 어려운 열도 있음 — 열이 있다고 모두 쓰지 않음

정상 가공과 이상 — 무엇이 다른가

정상은 고회전 유지 —회전수가 크게 떨어진 시점이 이상

40회 중 4회. 눈으로 셀 수 있는 양이지만 하루치는 수만 줄



그래서 배열 연산이 필요 —반복문 없이 조건 한 줄로 이 4건을 골라내는 것이 이 파트의 목표

PRACTICE

posco

실습 1. 온도 배열 만들고 화씨로 바꾸기

목표

측정한 섭씨 온도 묶음을 한 번에 화씨로 변환

단계

① 측정한 온도 여섯 개를 하나의 묶음으로 만들기

② 만든 묶음을 출력해 값이 어떻게 보이는지 확인

③ 묶음 전체에 1.8을 곱하고 32를 더해 화씨로 바꾸기

④ 바뀐 값을 출력해 여섯 개가 한 번에 변환됐는지 확인

예상 결과

섭씨 여섯 개가 화씨 여섯 개로 한 번에 바뀐 배열 출력

PRACTICE

실습 1. 온도 배열 만들고 화씨로 바꾸기 정답

반복문 없이 묶음 전체에 곱셈과 덧셈이 한 번에 적용된다

CODE

```
import numpy as np

# MCT 가공 온도 6회 측정 (실측값)
temp = np.array([30, 30, 30, 30, 29, 30])
print(temp)
# 출력: [30 30 30 30 29 30]
print(temp * 1.8 + 32)
# 출력: [86.  86.  86.  86.  84.2 86. ]
```

np.arange로 일정 간격 숫자 만들기

posco

시작부터 끝 직전까지 일정 간격으로 값 생성

CODE

```
print(np.arange(5))
# 출력: [0 1 2 3 4]
print(np.arange(0, 10, 2))
# 출력: [0 2 4 6 8]
```

np.arange(5)는 0부터 4까지 — 핵심은 **끝 숫자 5가 제외**된다는 점.

세 번째 값 2는 간격. 끝 숫자 제외는 arange만의 규칙이 아니라 파이썬 전체의 약속.

np.linspace로 같은 간격 값 만들기

posco

시작과 끝 사이를 같은 간격의 값 여러 개로 채우기

CODE

```
print(np.linspace(0, 1, 5))
# 출력: [0. 0.25 0.5 0.75 1. ]
```

세 번째 값 5는 간격이 아니라 만들 값의 개수 —값 5개면 구간은 4개.

arange는 간격을 정하지만 linspace는 개수를 정합니다.

→ 출력의 **끝값 1이 포함**되어 있음 (arange는 끝값 제외, linspace는 끝값 포함).

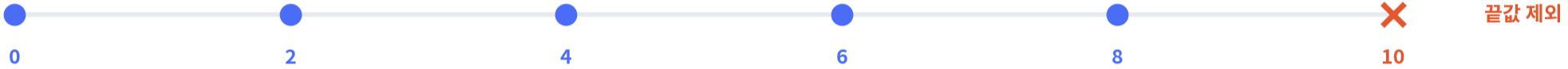
arange와 linspace — 무엇을 정하나

간격이 중요하면 arange, 개수가 중요하면 linspace

같은 구간이라도 "내가 무엇을 손에 쥐고 있는가"가 다르다

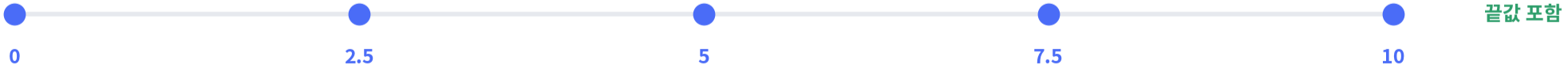
np.arange(0, 10, 2)

내가 정하는 것 = 간격 2 · 개수 5개는 세어 봐야 앎



np.linspace(0, 10, 5)

내가 정하는 것 = 개수 5개 · 간격 2.5는 자동 계산



간격을 알면 arange · 개수를 알면 linspace

그래프 축을 만들 때는 보통 개수를 정하니 **linspace**를 자주 씁니다.

zeros • ones • full 초기화 배열

posco

같은 값으로 가득 찬 배열을 한 번에 생성

CODE

```
print(np.zeros(3))
# 출력: [0. 0. 0.]
print(np.ones(3))
# 출력: [1. 1. 1.]
print(np.full(3, 7))
# 출력: [7 7 7]
```

np.zeros(3)은 0이 세 개 든 배열. 출력의 **0.**은 실수 배열이라는 표시.

np.full(3, 7)은 7이 세 개. np.ones는 1로, np.full은 정한 값으로 채웁니다.

초기화 함수 — 무엇으로 채우나

posco

용도에 따라 **0 • 1 • 특정 값**으로 채우는 세 가지 함수

np.zeros

0으로 채움.

결과를 담을 **빈 그릇**을 준비할 때.

np.ones

1로 채움.

비율이나 **가중치 출발점**으로.

np.full

특정 값으로 채움.

기준값을 설정할 때.

한 줄짜리 배열을 만드는 방법 —표 모양으로 만드는 방법은 뒤에서 다룸

배열 생성 함수 정리

posco

만들고 싶은 배열에 따라 **함수 선택**

함수	언제	예시
np.array	값을 알 때	센서 측정값
np.arange	간격 필요	측정 시점
np.linspace	개수를 정함	그래프 축
np.zeros	0으로 채움	빈 그릇
np.full	특정 값으로 채움	기준값 설정

값을 아는가 array • 간격을 정하는가 arange • 개수를 정하는가 linspace • 같은 값으로 채우는가 zeros•full

PRACTICE

posco

실습 2. 전류 구간을 같은 간격의 값으로 채우기

목표

전류가 움직이는 구간을 같은 간격의 값 여덟 개로 채우기

단계

① 전류가 0부터 70까지 움직인다고 보고 시작값과 끝값을 정하기

② 그 사이를 같은 간격의 값 여덟 개로 채우기

③ 나온 값을 출력해 이웃한 값의 차이가 모두 같은지 확인

④ 끝값 70이 결과에 들어 있는지 확인

예상 결과

0부터 70까지 같은 간격으로 놓인 값 여덟 개 출력

PRACTICE

posco

실습 2. 전류 구간을 같은 간격의 값으로 채우기 정답

개수를 정하면 간격은 자동으로 계산되고 끝값이 포함된다

CODE

```
import numpy as np

# 전류가 움직이는 구간을 같은 간격의 값 8개로
scale = np.linspace(0, 70, 8)
print(scale)
# 출력: [ 0. 10. 20. 30. 40. 50. 60. 70.]
```

PRACTICE

posco

실습 3. 측정 순번 배열 만들기

목표

측정 순번을 일정하게 건너뛰며 확인할 번호 묶음 생성

단계

① 측정이 모두 40번 있었다고 보고 시작 번호와 끝 번호를 정하기

② 다섯 번마다 한 번씩 확인하도록 건너뛸 폭을 정하기

③ 정한 값으로 번호 묶음을 만들어 출력

④ 끝 번호 40이 결과에서 빠졌는지 확인

예상 결과

0부터 5씩 증가하는 여덟 개 번호 배열 출력

PRACTICE

실습 3. 측정 순번 배열 만들기 정답

간격을 정하면 개수는 계산해야 알 수 있고, 끝값은 빠진다

CODE

```
import numpy as np

# 40회 측정 중 5회마다 확인할 순번
idx = np.arange(0, 40, 5)
print(idx)
# 출력: [ 0  5 10 15 20 25 30 35]   ← 40은 제외
```

1차원과 2차원 배열

배열은 모양에 따라 **1차원과 2차원**으로 나뉜다

1차원 — 방향이 하나

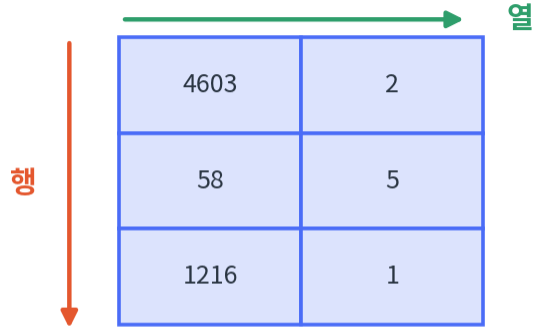


왼쪽 → 오른쪽, 순서 하나만 있음

shape (5,) · ndim 1

센서 하나가 시간에 따라 잰 값

2차원 — 방향이 둘



shape (3, 2) · ndim 2

행 = 시점, 열 = 센서

1차원은 줄 서 있는 사람 — 앞뒤 순서만 / 2차원은 극장 좌석 — 몇 번째 줄, 몇 번째 자리

차원은 값을 늘어놓는 방향의 수 — 설비 데이터는 보통 행= 시점, 열= 센서인 2차원 표.

배열의 차원(ndim)

posco

ndim으로 배열이 몇 차원인지 확인 — **괄호 없는 속성**

CODE

```
a = np.array([1, 2, 3])
b = np.array([[1, 2], [3, 4]])
print(a.ndim)
# 출력: 1
print(b.ndim)
# 출력: 2
```

ndim은 **n**umber of **dim**ensions, 차원 개수.

한 줄짜리는 **1**, 표 모양은 **2**. ndim뒤에 **괄호가 없다**는 것이 핵심.

속성과 함수의 차이

posco

> 함수는 동작을 실행 —**괄호 있음** (예: np.array())
>
> 속성은 정보를 꺼내봄 — **괄호 없음** (예: 배열.ndim)

무언가를 시키는 **명령(함수)**은 괄호가 붙습니다. `np.array()`는 "배열을 만들어라"라는 동작.

배열이 이미 가진 정보를 꺼내 보는 **속성**은 괄호가 없습니다.

차원·형태·크기 같은 정보는 배열이 만들어질 때 **이미 딸려 있어서**, 새로 계산하는 게 아니라 들여다보는 것.

→ 이 구분은 **shape, size, dtype**에도 똑같이 적용.

배열의 형태(shape)

posco

shape으로 행과 열 크기 확인 — **앞=행, 뒤=열**

CODE

```
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b.shape)
# 출력: (2, 3)
```

2행 3열 배열의 shape이 괄호로 묶인 **(2, 3)**으로 나옵니다.

앞 숫자 **2가 행**, 뒤 숫자 **3이 열**. 이 순서는 2차원 인덱싱에서도 똑같이 쓰입니다.

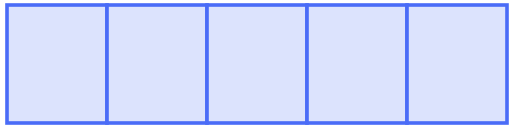
괄호로 묶여 나오는 건 결과가 **튜플**이기 때문.

shape 읽는 법

분석가가 데이터를 처음 만나면 가장 먼저 던지는 질문 — **shape이 뭐지?**

(5,)

값 5개인 1차원



← 값 5개 한 줄 →

쉼표는 오타가 아니라
값 하나짜리 묶음이라는 표시

(2, 3)

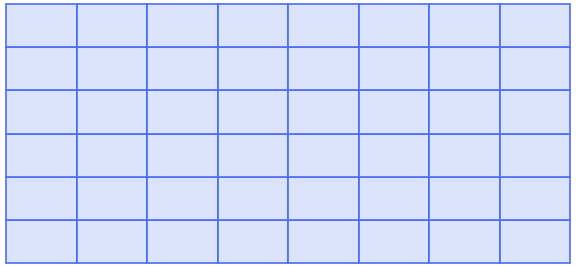
2행 3열



앞이 행, 뒤가 열
숫자 두 개 = 2차원

(1000, 20)

시점 1000 · 센서 20

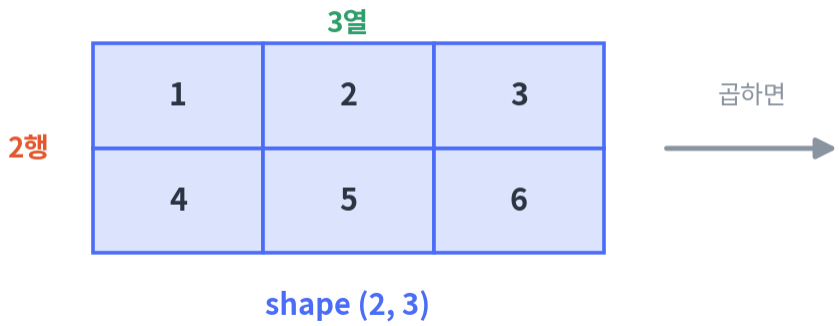


⋯ 계속 ⋯

실무 데이터는 대개
(시점 수, 센서 수)

배열의 크기(size)

size로 값의 전체 개수 확인 — **행 × 열**



곱하면

$$2 \times 3 = 6$$

size = 표 안의 칸 총 개수

shape은 "어떤 모양인가" · size는 "값이 모두 몇 개인가"

1차원이면



shape (5,) → size 5

len()은 행 수만 돌려주므로 주의

2행 3열 배열의 size는 **2 × 3 = 6**. 표 안의 **칸 총 개수**.

shape은 "어떤 모양인가", **size**는 "값이 모두 몇 개인가".

size도 속성이라 괄호가 없음 — shape의 숫자들을 모두 곱하면 size이고, 모양을 바꿔도 총 개수는 그대로.

배열의 크기(size)

posco

size로 값의 전체 개수 확인 — **행 × 열**

CODE

```
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b.size)
# 출력: 6
```

자료형(dtype)

dtype으로 값의 종류 확인 —**정수 int, 실수 float**

CODE

```
a = np.array([1, 2, 3])
b = np.array([1.5, 2.7, 3.1])
print(a.dtype)
# 출력: int64
print(b.dtype)
# 출력: float64
```

정수는 **int**, 실수는 **float**. 뒤의 **64**는 저장 공간 크기.

dtype을 꼭 확인해야 하는 이유

데이터를 불러온 직후 **dtype 확인**이 계산 오류를 막아줌

정수 int

소수점 없는 값.

측정 순번, 설비 번호.

실수 float

소수점 있는 값.

온도, 진동.

숨은 글자

숫자처럼 보여도 **글자로 저장**되어 있을 수 있음.

계산 시 **오류 발생**.

형변환(astype)

astype으로 자료형 바꾸기 — 실수→정수는 **소수점 버림**

CODE

```
a = np.array([1.7, 2.3, 3.9])
print(a.astype(int))
# 출력: [1 2 3]
```

실수 배열에 **astype(int)**를 하면 정수로 바뀝니다.

꼭 짚어야 할 점 — **1.7이 2가 아니라 1**이 됩니다. 1.9도 1.

→ **반올림이 아니라 소수점 아래를 그냥 버립니다**.

astype 사용 시 주의점

실수를 정수로 바꾸면 **소수점 아래를 버림** (1.9 → 1)

원본은 그대로, 새 배열이 생김 —**결과를 새 이름에 담기**

99.9를 정수로 바꾸면 100이 아니라 99가 됩니다.

정밀한 측정값은 함부로 정수로 바꾸지 않는 게 좋습니다.

astype는 변환된 **새 배열을 만들 뿐 원본은 그대로**입니다.

결과를 계속 쓰려면 `z = a.astype(int)`처럼 새 이름에 담아야 합니다.

→ 그냥 실행만 하고 안 담으면 **결과가 사라집니다**.

배열 속성 종합 정리

데이터를 만나면 **가장 먼저 확인하는 네 가지**

속성	알려주는 것	괄호
ndim	차원 개수	없음
shape	행·열 크기	없음
size	값 총 개수	없음
dtype	자료형	없음

shape의 숫자 개수가 **ndim**이고, shape을 곱하면 **size**. dtype만 모양과 무관한 독립 정보.

데이터 구조 파악 3단계

새 데이터를 만났을 때 따라야 할 순서 —**운전 전 거울·계기판 확인과 같음**

STEP 01

ndim으로 차원 확인

1차원? 2차원? 이후 작업 방향을 정함

STEP 02

shape으로 행•열 크기 파악

데이터 규모를 한눈에

STEP 03

size로 양, dtype으로 자료형 점검

계산 가능한 상태인지 확인

PRACTICE

posco

실습 4. 두 센서 표의 구조 확인하기

목표

시점별 온도•전류 표의 차원•형태•개수를 속성으로 확인

단계

① 네 시점의 온도와 전류를 시점마다 한 줄씩 묶어 표 모양으로 준비

② 이 묶음이 몇 방향으로 늘어서 있는지 확인

③ 몇 줄 몇 칸인지 확인

④ 값이 모두 몇 개인지 확인

⑤ 줄 수와 칸 수를 곱한 값이 전체 개수와 같은지 확인

예상 결과

차원 2, 형태 (4, 2), 개수 8 출력

PRACTICE

posco

실습 4. 두 센서 표의 구조 확인하기 정답

줄 수 × 칸 수 = 전체 개수 — shape과 size는 연결되어 있다

CODE

```
import numpy as np

# 4시점 × 2센서(온도, 전류)
data = np.array([[30, 15.44], [30, 18.69],
                 [30, 62.48], [30, 18.37]])
print(data.ndim)    # 2
print(data.shape)   # (4, 2)
print(data.size)    # 8   ← 4 × 2
```

PRACTICE

posco

실습 5. 전류 자료형 확인하고 정수로 바꾸기

목표

소수점이 있는 전류값의 자료형을 확인하고 정수로 변환

단계

① 소수점이 있는 전류 측정값 네 개를 묶음으로 준비

② 이 묶음이 어떤 종류의 값을 담고 있는지 확인

③ 소수점을 없앤 정수 묶음으로 바꿔 새 이름에 담기

④ 바뀐 값을 출력해 소수점 아래가 반올림이 아니라 잘려 나갔는지 확인

예상 결과

자료형 float64 확인, 18.69가 19가 아닌 18로 잘린 정수 배열 출력

PRACTICE

실습 5. 전류 자료형 확인하고 정수로 바꾸기 정답

18.69 → 18 · 62.48 → 62 — 반올림이 아니라 버림

CODE

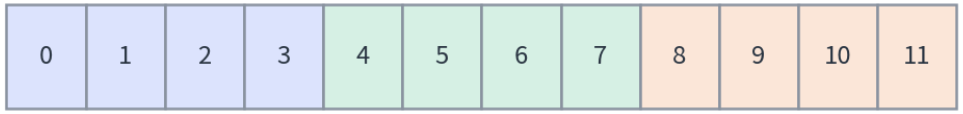
```
import numpy as np

# 전류 4회 측정 (실측값)
amp = np.array([15.44, 18.69, 62.48, 18.37])
print(amp.dtype)          # float64
cut = amp.astype(int)
print(cut)                # [15 18 62 18]
```

reshape로 형태 바꾸기

값은 그대로, **모양만 바꾸기** —값 개수는 같아야 함

한 줄 배열 — 값 12개



np.arange(12) · shape (12,)

reshape(3, 4)

표 모양 — 3행 4열 (값은 그대로)

0행	0	1	2	3
1행	4	5	6	7
2행	8	9	10	11

색이 같은 네 칸이 한 줄이 됨 — 값 개수 12는 3 × 4와 맞아야 함

값 12개짜리를 **3행 4열**로 바꿈. 값은 그대로고 모양만 변함.

-1을 쓰면 NumPy가 나머지 크기를 자동 계산.

reshape로 형태 바꾸기

posco

값은 그대로, **모양만 바꾸기** — 값 개수는 같아야 함

CODE

```
a = np.arange(12)
print(a.reshape(3, 4))
# 출력: 3행 4열 배열 (0~11)
print(a.reshape(-1, 2))
# 출력: 6행 2열 (행은 자동 계산)
```

값 12개짜리를 **3행 4열**로 바꿈. 값은 그대로고 모양만 변함.

-1을 쓰면 NumPy가 나머지 크기를 자동 계산.

reshape의 규칙

바꾸기 전후 **값 개수가 같아야** 함 — 12개를 3행 5열로는 불가

행이나 열에 **-1**을 넣으면 나머지를 자동 계산

12개는 3행 4열이나 2행 6열은 되지만, 3행 5열은 15칸이라 안 됩니다.

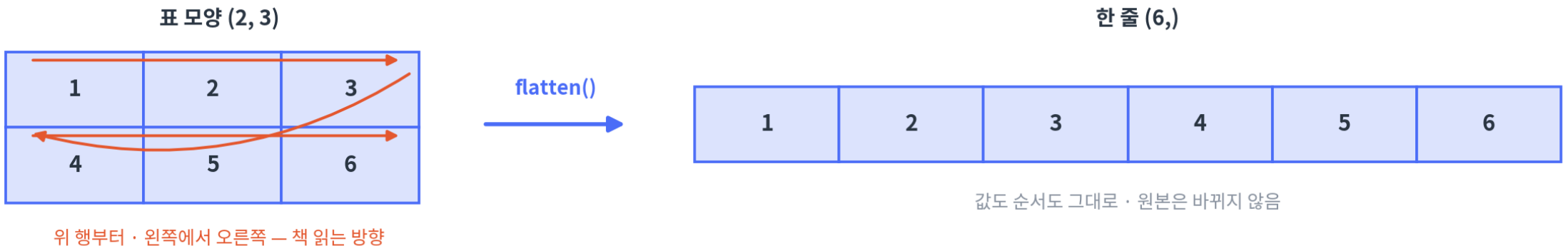
`cannot reshape` 오류는 거의 이 문제.

데이터가 많아 개수를 세기 어려울 때, **열만 정하고 행은 -1**로 두면 NumPy가 맞춤.

→ 단, -1은 **행과 열 중 한 곳에만** 쓸 수 있습니다.

flatten으로 1차원 만들기

표 모양을 **한 줄로 펴기** — 위 행부터 왼쪽에서 오른쪽으로



reshape이 한 줄을 표로 만든다면, **flatten은 반대로 표를 한 줄로 폅니다**.

펴지는 순서: **위 행부터, 그 안에서 왼쪽에서 오른쪽** — 책 읽는 방향.

첫 행부터 차례로 이어 붙임 — 센서 구분 없이 표 전체로 통계를 낼 때 유용하며 원본은 그대로.

flatten으로 1차원 만들기

posco

표 모양을 **한 줄로 펴기** — 위 행부터 왼쪽에서 오른쪽으로

CODE

```
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b.flatten())
# 출력: [1 2 3 4 5 6]
```

reshape이 한 줄을 표로 만든다면, **flatten은 반대로 표를 한 줄로 폅니다**.

펴지는 순서: **위 행부터, 그 안에서 왼쪽에서 오른쪽** — 책 읽는 방향.

첫 행부터 차례로 이어 붙임 — 센서 구분 없이 표 전체로 통계를 낼 때 유용하며 원본은 그대로.

형태 변환 정리

posco

모양을 바꾸는 두 함수 — **값은 그대로, 배치만 변경**

함수	하는 일	결과
reshape	모양 바꾸기	원하는 행·열
flatten	한 줄로 펴기	1차원 배열

공통점 —값 자체는 바꾸지 않고 **배치만 바꾸며, 값의 총 개수는 변하지 않음**.

한 줄로 들어온 데이터를 표로 정리할 땐 **reshape**, 표를 한 줄로 펴서 전체를 다룰 땐 **flatten**.

→ 둘 다 결과를 **새 이름에 담아야** 쓸 수 있습니다.

PRACTICE

posco

실습 6. 사이클타임을 표 모양으로 정리하기

목표

한 줄로 이어진 열두 번의 측정값을 표 모양으로 변환

단계

① 열두 번 측정한 사이클타임을 한 줄 묶음으로 준비

② 열두 개가 몇 줄 몇 칸이면 딱 맞는지 정하기

③ 정한 줄•칸 수로 모양을 바꾸기

④ 바뀐 묶음을 출력해 값이 하나도 늘거나 줄지 않았는지 확인

예상 결과

열두 개 값이 3행 4열 표 모양으로 바뀐 배열 출력

PRACTICE

posco

실습 6. 사이클타임을 표 모양으로 정리하기 정답

값 개수 12가 3 × 4와 맞아야 모양을 바꿀 수 있다

CODE

```
import numpy as np

# 사이클타임 12회 측정 (실측값)
ct = np.array([21156, 15611, 15633, 15601,
               15607, 15557, 15557, 15636,
               15600, 15634, 21207, 15554])
print(ct.reshape(3, 4))
# [[21156 15611 15633 15601]
#  [15607 15557 15557 15636]
#  [15600 15634 21207 15554]]
```

PRACTICE

posco

실습 7. 세 센서 표로 정리하고 다시 펴기

목표

한 줄로 이어진 측정값을 행=시점, 열=센서 표로 정리한 뒤 되돌리기

단계

① 네 시점 동안 잰 온도·전류·회전수를 시점 순서대로 한 줄로 이어 준비

② 한 줄이 한 시점이고 한 칸이 한 센서가 되도록 모양을 바꾸기

③ 바뀐 표를 출력해 각 줄이 한 시점의 세 센서값인지 확인

④ 표를 다시 한 줄로 펴서 처음 순서와 같은지 확인

예상 결과

열두 개 값이 4행 3열 표로 정리되고, 다시 펴면 처음 순서로 돌아옴

PRACTICE

posco

실습 7. 세 센서 표로 정리하고 다시 펴기 정답

모양은 바뀌어도 값과 그 순서는 그대로 유지된다

CODE

```
import numpy as np

# 4시점 × 3센서(온도, 전류, 회전수)를 한 줄로 이어 놓은 상태
flat = np.array([30, 15.44, 4603, 30, 18.69, 4603,
                 30, 62.48, 4967, 30, 18.37, 4603])
table = flat.reshape(4, 3)   # 행=시점, 열=센서
print(table)
# [[  30.     15.44 4603.  ] ... 4줄
print(table.flatten())       # 처음 순서 그대로 12개
```

CLOSING

데이터 분석의 기본 흐름

분석은 항상 **만들기 → 구조 확인 → 모양 정리** 순서

이 흐름이 몸에 배면 **어떤 데이터도 다룰 수 있음**

STEP 01

만들기

np.array

STEP 02

구조 확인

ndim · shape · size · dtype

STEP 03

모양 정리

reshape · flatten